Practical no. 15

Aim - Screenshot Capture and Logging Mechanism

Implement screenshot capture on test failure and integrate logging mechanism using Log4j. Generate execution logs and analyze error messages.

Step 1: Create Maven Project

In Eclipse: File → New → Maven Project → Create

Group Id: com.selenium

Artifact Id: ScreenshotLogging

Version: 0.0.1-SNAPSHOT

Packaging: jar

Project name: ScreenshotLogging

Step 2: Add Dependencies

In pom.xml, add Selenium and Log4j dependencies.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.selenium</groupId>
    <artifactId>ScreenshotLogging</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
    </properties>
    <dependencies>
        <!-- Selenium WebDriver -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.35.0</version>
        </dependency>
        <!-- Log4j API -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>2.25.1</version>
        </dependency>
        <!-- Log4j Core -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>2.25.1</version>
        </dependency>
    </dependencies>
</project>
```

In Eclipse: Ctrl + S

Then: Right-click project → Maven → Update Project → check your project → OK

Wait for Maven to finish downloading the dependencies.

Step 3: Create Log4j Configuration (log4j2.xml)

Right-click src/test/resources → New → File

Enter: log4j2.xml (src/test/resources/log4j2.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Configuration status="WARN">
    <Appenders>
        <!-- Console Appender -->
        <Console name="Console">
            <PatternLayout
                pattern="%d{HH:mm:ss} %-5level %logger{36} - %msg%n"/>
        </Console>
        <!-- File Appender -->
        <File name="File" fileName="logs/test.log">
            <PatternLayout
                pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level - %msg%n"/>
        </File>
    </Appenders>
    <Loggers>
        <Root level="info">
            <AppenderRef ref="Console"/>
            <AppenderRef ref="File"/>
        </Root>
    </Loggers>
</Configuration>
```

Step 4: Create Test Class

Right-click src/test/java → New → Class

Class name: ScreenshotLoggingTest → Finish

```
import java.io.File;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.By;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class ScreenshotLoggingTest {

    static Logger log = LogManager.getLogger(ScreenshotLoggingTest.class);
    static WebDriver driver;

    public static void main(String[] args) {
        try {
            // Start test
            log.info("Test execution started");

            // Launch Chrome
            driver = new ChromeDriver();
            log.info("Chrome browser launched");

            // Open Google
            driver.get("https://www.google.com");
            log.info("Google website opened");

            // Intentional failure
            log.info("Searching for wrong element");
            driver.findElement(By.id("wrongElement")).click();

            // This line will not execute because the above line fails
            log.info("Test completed successfully");

        } catch (Exception e) {
            // Log error
            log.error("Test failed: " + e.getMessage());

            // Capture screenshot
            takeScreenshot();

        } finally {
            // Close browser
            if (driver != null) {
                driver.quit();
                log.info("Browser closed");
```

```
            }
        }
    }

    public static void takeScreenshot() {
        try {
            TakesScreenshot ts = (TakesScreenshot) driver;
            File source = ts.getScreenshotAs(OutputType.FILE);
            File destination = new File("screenshots/test_failure.png");

            // Create screenshots folder if it does not exist
            destination.getParentFile().mkdirs();

            // Save screenshot
            if (source.renameTo(destination)) {
                log.info("Screenshot captured: "
                        + destination.getAbsolutePath());
            } else {
                log.error("Screenshot could not be saved.");
            }
        } catch (Exception e) {
            log.error("Screenshot capture failed: "
                    + e.getMessage());
        }
    }
}
```

Step 5: Run the Program

Right-click ScreenshotLoggingTest.java → Run As → Java Application

Step 6: Analyze Log

Check the console output and logs/test.log for the execution log and error message.